zkDietFormer

Anonymous Author(s)

Abstract

Keywords

Zero-Knowledge Proofs

1 Introduction

Recent breakthroughs in Transformer-based models have revolutionized artificial intelligence, facilitating the widespread deployment of high-performance Large Language Models (LLMs) via Model-as-a-Service (MaaS) platforms. In this setting, model providers typically large models, while users receive outputs through APIs or remote inference interfaces. However, this architecture inherently obscures the generation process: users cannot independently verify whether the received output truly produced from the specific model architecture and weight parameters intended. Users are therefore forced to rely on trust regarding whether the provider substituted a cheaper model for the same input, post-processed or modified outputs due to internal policies, or unintentionally changed the model during updates or deployment. This problem of model execution integrity is particularly critical in high-stakes domains such as finance, healthcare, and security, and becomes increasingly central as demands for transparency and accountability in inference continue to grow.

One may attempt to address this trust issue by requiring the model provider to disclose model parameters for verification. In practice, however, this is practically infeasible. Model weights are the result of substantial investments in data, compute, and engineering, and are protected as proprietary intellectual property (IP). Consequently, a fundamental dilemma arises: users demand integrity guarantees for the model's outputs, while providers must preserve the privacy of their model parameters to protect their assets.

Zero-knowledge proofs (ZKPs) provide a principled way to reconcile these conflicting requirements. By leveraging ZKPs, a prover (the model provider) can cryptographically prove that an output y is the result of evaluating the model on the user's input x with a secret weight W, i.e., $f(x, W) = y$, without revealing any information about W itself. This enables verifiable inference while maintaining the confidentiality of model parameters.

However, making ZKP-based verifiable AI practically deployable requires the efficient proving of computationally intensive deep learning operations. In modern Transformer architectures, the primary bottleneck is the self-attention mechanism, the computational core of the model. For an input sequence of length n and hidden dimension d, the input $X \in \mathbb{F}^{n\times d}$ is projected into queries, keys, and values $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{F}^{n\times d}$. Standard self-attention forms an attention matrix $\mathbf{QK}^\top \in \mathbb{F}^{n\times n}$, applies a row-wise softmax over the entire matrix, and then multiplies by $\mathbf{V}$ to obtain the attention output.

This computation requires $O(n^2 d)$ linear operations for matrix multiplications, as well as an additional $O(n^2)$ nonlinear cost induced by the softmax. The resulting computational overhead is substantial, posing the single largest obstacle to proving large-scale Transformer models.

The most naïve approach to proving self-attention involves arithmetizing the full $\mathbf{QK}^\top$ computation into a circuit and enforcing the resulting $O(n^2 d)$ operations within a SNARK. However, this cost far exceeds practical limits. To mitigate this, many systems have adopted probabilistic verification techniques for large matrix products, such as Freivalds' algorithm [11] or Thaler's randomized checks [23]. These methods verify the $n \times n$ multiplication relation without full recomputation by sampling random vectors, compressing verification to a small number of checks and reducing the cost to $O(n^2)$. Nevertheless, the complexity remains dominated by the quadratic term, which becomes increasingly prohibitive as models scale. For instance, while BERT [9] uses 512 tokens and GPT-3 [2] uses a 2,048-token context window, frontier models now support contexts in the hundreds of thousands or even millions (e.g., 400K for GPT-5 [19] and 10M for Llama 4 Scout [18]). As n grows, this quadratic verification cost renders proof generation for realistic sequence lengths impractical.

Meanwhile, deep learning literature has repeatedly demonstrated that attention matrices often exhibit an approximate low-rank structure. Concretely, when $k \ll n$, the attention matrix can be effectively approximated by a rank-k representation, enabling training and inference costs to approach linearity with respect to the sequence length [25, 29]. Despite these advances, the field of Verifiable AI has yet to exploit low-rank structures as a primary mechanism for reducing proof generation costs. Consequently, a gap persists: while low-rank approximations improve computational efficiency in practice, the lack of proof systems tailored to these structures keeps proof complexity anchored at $O(n^2)$. This work aims to bridge this gap. Our central idea is to combine low-rank structures with probabilistic verification to eliminate the quadratic complexity in verifying self-attention matrices.

1.1 Our Contribution

In this work, we propose **zkDietFormer**, a verification protocol that eliminates the dominant $O(n^2)$ bottleneck in proving self-attention by combining a low-rank representation of the attention matrix, Freivalds' randomized checks, and a commit-and-prove SNARK. Our main contributions are summarized as follows:

- **The first verification framework tailored to low-rank self-attention.** To the best of our knowledge, we present the first verification framework specifically designed for

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Proceedings on Privacy Enhancing Technologies YYYY(X), 1–13

© YYYY Copyright held by the owner/author(s).

https://doi.org/XXXXXXX.XXXXXXX

low-rank self-attention. We instantiate this framework using the Nyström method, reducing the overall circuit complexity—including both the core matrix operations of self-attention and the nonlinear operations induced by row-wise softmax—to $O(n)$.

- **Constraint-efficient verification via Freivalds' checks and CP-SNARKs.** We employ Freivalds' algorithm to efficiently verify large matrix computations inside the circuit, while leveraging commit-and-prove SNARKs to offload computationally expensive operations outside the circuit, significantly reducing the total number of constraints.
- **Practical implementation and evaluation.** We implemented a prototype of the proposed framework and evaluated its performance through benchmarks. The results demonstrate a significant reduction in prover costs for practical sequence lengths, showing that **zkDietFormer**is not only theoretically efficient but also practically deployable.

In summary, the prover's constraint cost in **zkDietFormer**scales linearly with the input sequence length n. Consequently, to the best of our knowledge, this work provides the first verifiable mechanism for low-rank self-attention that enables proof generation with cost linear to the sequence length.

2 Related Works

In this section, we provide a brief overview of the core operations of Transformer models and review prior work aimed at improving the efficiency of matrix multiplication, which is widely recognized as a major computational bottleneck. We also examine existing approaches for constructing zero-knowledge proofs for matrix multiplication.

Low-Rank Transformer The Transformer is a deep learning architecture proposed by Google in 2017 [24], which leverages the attention mechanism as its core component to effectively model relationships within sequential data, such as words in a sentence. The core computation of this model consists of matrix multiplications among the query, key, and value matrices. In standard Transformers, these operations incur quadratic computational complexity with respect to the input sequence length n, causing the computational cost to grow rapidly as n increases. To mitigate this bottleneck, Child et al. [6] proposed the Sparse Transformer, which restricts attention to predefined sparse patterns, using strided attention for data with periodic structures such as images and audio, and fixed attention for text without structure. However, this approach relies on implementations tied to specific TensorFlow versions. To address this limitation, Longformer [1] introduced sliding-window attention, where each token attends only to nearby tokens, providing a more general and maintainable solution. Rae et al. [20] proposed the Compressive Transformer, which compresses past hidden activations(memories) using pooling operations and jointly attends to both recent memories and compressed long-term memories to efficiently model long-range context. Nevertheless, these approaches do not preserve full attention, leading to limitations such as insufficient modeling of long-range dependencies [1, 6], or performance degradation in exchange for computational efficiency [1]. Accordingly, Linformer [25] shows that the attention matrix induced by self-attention exhibits a low-rank structure both theoretically and empirically, and approximates scaled dot-product attention by projecting the key and value matrices into a lower-dimensional space via linear projections. As a result, the time and space complexity of self-attention are reduced to linear in the sequence length n. However, Linformer has limitations in causal modeling, where future tokens must be masked, and can incur large mean squared errors compared to exact attention. To overcome these issues, Performer [7] introduces a linear-complexity attention mechanism that avoids explicit projection matrices and low-rank assumptions, and instead approximates full softmax attention via FAVOR+, which is based on positive orthogonal random features for softmax kernel estimation. Nyströmformer [29] introduces a Nyström low-rank approximation that reconstructs the full attention map using a subset of landmark tokens, achieving higher accuracy than Performer while maintaining linear complexity with respect to the sequence length.

zk-SNARKs for Matrix Multiplication There has been ongoing research on verifying the integrity of matrix multiplication using zero-knowledge proofs. Since convolution operations can be reduced to structured matrix multiplication, such verification techniques have been extensively studied as a core technique for proving the correctness and integrity of AI model execution.

Thaler et al. [23] introduced the reuse-of-work technique into the Sumcheck protocol for matrix multiplication verification, reducing the prover's computational overhead from $O(n^3)$ to $O(n^2)$ for $n \times n$ matrices. zkCNN [17] adopts this optimized Sumcheck protocol to efficiently verify matrix multiplication in fully connected layers. pvCNN [26] transforms convolution operations, which inherently require a large number of multiplication gates, into a single matrix multiplication by reshaping the filters and inputs into rectangular matrices. Inspired by the QPP (Quadratic Polynomial Programs) approach of vCNN [16], which applies polynomial representations within the QAP framework, pvCNN further extends this idea by introducing the QMP (Quadratic Matrix Program). As a result, the convolution can be expressed and verified using only a single multiplication gate. zkML [5] applies Freivalds' algorithm [11] to verify matrix multiplication operations in linear layers, such as convolutions and batched matrix multiplications, achieving verification complexity of $O(n^2)$. zkLLM [22] leverages the dedicated sumcheck protocol proposed in SafetyNets [12] to verify matrix multiplication operations arising in Transformer models. As a result, the prover's computational complexity scales linearly with the sizes of the input matrices, achieving a complexity of $O(mn + np)$. zkMatrix [8] reduces matrix multiplication to four inner-product relations and applies Bulletproofs-based verification acceleration techniques, thereby achieving a prover time of $O(n^2)$ and a verifier time of $O(\log n)$ for $n \times n$ committed matrix multiplication. zkVC [30] introduces Constraint-reduced Polynomial Circuit(CRPC) and the Prefix-Sum Query(PSQ) technique to significantly reduce the constraint complexity of matrix multiplication from $O(n^3)$ to $O(n)$.

However, these approaches still require a prover computation complexity of $O(n^2)$ with respect to the matrix dimension (or sequence length) [5, 8, 16, 17, 23], or, even when achieving linear-time complexity in theory, suffer from the limitation that the other matrix dimension m remains prohibitively large in practice [22]. Moreover, nonlinear functions such as softmax cannot be directly handled within ZKP circuits and thus must be approximated, which leads to

accuracy degradation and necessitates the design of model architectures specifically tailored for zero-knowledge proof systems [30].

3 Preliminaries

3.1 Scaled Dot-Product Attention

The model represents interactions between tokens using queries and keys of dimension d_k, and values of dimension d_v. For each token, its associated query computes its similarity scores with the keys of all other tokens in the sequence, thereby determining which parts of the input the model should attend to more strongly. The contextual information is then aggregated by computing a weighted sum of the corresponding value vectors based on these similarities.

The attention mechanism is defined as a function that takes a single query together with a set of key–value pairs and produces an output vector, where the query, keys, values, and outputs all lie in vector spaces. In practice, multiple queries are processed simultaneously by stacking them into matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. These matrices are obtained by projecting the input matrix $\mathbf{X}$ using learnable parameter matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$, such that $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, and $\mathbf{V} = \mathbf{X}\mathbf{W}^V$. Concretely, the attention weights are obtained by scaling the dot product between queries and keys by $1/\sqrt{d_k}$, followed by the application of the softmax function. The final output is computed as the weighted sum of the value vectors using these attention weights.

The output matrix is computed as follows:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

3.2 Multihead Attention

Multi-head attention replaces a single attention function over d_{model}-dimensional queries, keys, and values with h parallel attention heads. The queries, keys, and values are first linearly projected h times into lower-dimensional subspaces of sizes d_q, d_k, and d_v, respectively. For each head i,

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$$

where $\mathbf{Q}_i \in \mathbb{R}^{n\times d_q}, \mathbf{K}_i \in \mathbb{R}^{n\times d_k}$ and $\mathbf{V}_i \in \mathbb{R}^{n\times d_v}$. The resulting head outputs are concatenated and linearly transformed to form the final representation:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O$$

with $\mathbf{W}^O \in \mathbb{R}^{hd_v\times d_{\text{model}}}$. This design enables the model to attend to diverse representation subspaces while maintaining computational cost comparable to single-head attention through dimensionality reduction within each head.

3.3 Nystrom-Based Linear Attention

Nyströmformer applies the Nyström method []—a technique widely used in matrix approximation—to reduce the computational complexity of the standard Transformer from $O(n^2)$ to $O(n)$. The key idea of the Nyström method is to reconstruct the full matrix approximately by using only a subset of its rows and columns, instead of computing the entire matrix explicitly.

When the softmax matrix S is expressed as a block matrix as shown below,

$$S = \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}) = \begin{bmatrix} A_s & B_s \\ F_s & C_s \end{bmatrix}$$

the Nystrom method approximates the expensive block $C_S \in \mathbb{R}^{(n-m)\times(n-m)}$ by using the smaller blocks $A_S \in \mathbb{R}^{m\times m}, B_S \in \mathbb{R}^{m\times(n-m)}, F_S \in \mathbb{R}^{(n-m)\times m}$ as illustrated below:

$$\hat{S} = \begin{bmatrix} A_S & B_S \\ F_S & F_S A_S^+ B_S \end{bmatrix} = \begin{bmatrix} A_S \\ F_S \end{bmatrix} A_S^+ \begin{bmatrix} A_S & B_S \end{bmatrix}$$

Here, A_S^+ denotes the Moore-Penrose pseudoinverse of A_S.

However, when this approach is applied directly to self-attention, the row-wise normalization property of the softmax function still requires accessing all entries of the $n\times n$ matrix in order to compute the denominators. To address this issue, Nyströmformer partitions the entire sequence into m segments before applying softmax, and computes the mean within each segment to obtain m landmark points for the queries and keys, respectively. These landmarks are then used to construct the $n \times m$ matrix $\tilde{\mathbf{Q}}$ and the $m \times n$ matrix $\tilde{\mathbf{K}}$, respectively. Using these matrices, Nyströmformer derives a final approximation that achieves linear computational complexity.

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\tilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)\text{softmax}\left(\frac{\tilde{\mathbf{Q}}\tilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)^+\text{softmax}\left(\frac{\tilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)V$$

The exact computation of the pseudoinverse A_S^+ (e.g., via SVD) is computationally expensive and poorly suited for GPU execution, becoming a practical bottleneck in implementation. To mitigate this issue, Nyströmformer introduces an iterative algorithm that approximates the pseudoinverse using repeated matrix multiplications. The resulting approximation Z^* is then substituted into the final formulation, significantly improving computational efficiency.

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\tilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)Z^*\,\text{softmax}\left(\frac{\tilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)V$$

3.4 zk-SNARKs

A *zero-knowledge succinct non-interactive argument of knowledge*(zk-SNARKs) for an NP relation $\mathcal{R}$ consists of three PPT algorithms Π = (Setup, Prove, Verify):

- $(\text{pk}, \text{vk}) \leftarrow \Pi.\text{Setup}(1^\lambda, \mathcal{R})$: On input the security parameter λ and a relation $\mathcal{R}$, the setup algorithm outputs a proving key pk and a verification key vk.
- $\pi \leftarrow \Pi.\text{Prove}(\text{pk}, \mathbf{x}; \mathbf{w})$: On input the proving key pk, a statement $\mathbf{x}$, and a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, the prover outputs a proof π.
- $0/1 \leftarrow \Pi.\text{Verify}(\text{vk}, \mathbf{x}, \pi)$: On input the verification key vk, a statement $\mathbf{x}$, and a proof π, the verifier outputs 0(reject) or 1(accept)

A zk-SNARK system Π satisfies the following properties:
Completeness. For every true statement for the relation $\mathcal{R}$, an honest prover holding a valid witness always convinces the verifier, i.e., $\forall(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$:

$$\Pr\left[\begin{matrix}(\text{pk}, \text{vk}) \leftarrow \Pi.\text{Setup}(1^\lambda, \mathcal{R}) \\ \pi \leftarrow \Pi.\text{Prove}(\text{pk}, \mathbf{x}; \mathbf{w})\end{matrix} : \quad \Pi.\text{Verify}(\text{vk}, \mathbf{x}, \pi) = 1\right] = 1$$

Knowledge Soundness. Knowledge soundness requires that any prover producing a proof π that makes the verifier accept must possess a corresponding witness, and that a knowledge extractor $\mathcal{E}$ can efficiently extract such a witness from π. Formally, for any PPT adversary $\mathcal{A}$, the following is *negligible*.

$$\Pr\left[\begin{array}{lll} (\mathsf{pk},\mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(\mathcal{R}) & & \Pi.\mathsf{Verify}(\mathsf{vk},\mathbf{x}^*,\pi^*)=1 \\ (\mathbf{x}^*,\pi^*) \leftarrow \mathcal{A}(\mathsf{pk},\mathsf{vk}) & : & \wedge \\ \mathbf{w} \leftarrow \mathcal{E}_{\mathcal{A}}(trans_{\mathcal{A}}) & & (\mathbf{x}^*;\mathbf{w}) \notin \mathcal{R} \end{array}\right]$$

Zero-Knowledge. There exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that $\mathcal{S}_1$ outputs simulated setup parameters $(\mathsf{pk}, \mathsf{vk})$ together with a trapdoor td. On input $(\mathsf{pk}, \mathsf{vk}, td)$, $\mathcal{S}_2$ outputs a simulated proof π. Moreover, for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have that

$$\left|\Pr\left[\mathcal{A}_2(\mathsf{vk},\mathbf{x},\pi)=1 : \begin{array}{l}(\mathsf{pk},\mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(1^\lambda,\mathcal{R}) \\ (\mathbf{x},\mathbf{w}) \leftarrow \mathcal{A}_1(1^\lambda,\mathsf{pk},\mathsf{vk}) \\ \pi \leftarrow \Pi.\mathsf{Prove}(\mathsf{pk},\mathbf{x};\mathbf{w})\end{array}\right]\right.$$
$$\left.-\Pr\left[\mathcal{A}_2(\mathsf{vk},\mathbf{x},\pi)=1 : \begin{array}{l}(\mathsf{pk},\mathsf{vk},td) \leftarrow \mathcal{S}_1(1^\lambda,\mathcal{R}) \\ (\mathbf{x},\mathbf{w}) \leftarrow \mathcal{A}_1(1^\lambda,\mathsf{pk},\mathsf{vk}) \\ \pi \leftarrow \mathcal{S}_2(\mathsf{pk},\mathbf{x},td)\end{array}\right]\right| \le \mathsf{negl}(\lambda).$$

Succinctness. For any statement $\mathbf{x}$ and witness $\mathbf{w}$, the size of proof π satisfies $|\pi| = poly(\lambda) \cdot polylog(|\mathbf{x}| + |\mathbf{w}|)$

Commit-and-Prove SNARKs (CP-SNARKs). A *Commit-and-Prove SNARKs* (CP-SNARKs) [3, 4] allow a prover to prove statements about values hidden in a commitment. Let Com be a commitment scheme. Given a commitment cm to $\mathbf{u}$, a CP-SNARK proves knowledge of a witness $\mathbf{w}$ consistent with cm and satisfying a relation $\mathcal{R}$. Formally, for $\mathcal{R}(\mathbf{x};\mathbf{w})$, we write $\mathbf{w} := (\omega, \mathbf{u}, o)$, where $\mathbf{u}$ is committed, ω is uncommitted, and o is the opening randomness, such that $\mathsf{cm} = \mathsf{Com}(\mathbf{u}; o)$ and $\mathcal{R}(\mathbf{x};\omega) = 1$. A CP-SNARK is a tuple of algorithms $\Pi_{\mathsf{cp}} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$.

- $(\mathsf{pk},\mathsf{vk}) \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Setup}(1^\lambda, \mathcal{R}, \mathsf{ck})$: Same as $\Pi.\mathsf{Setup}$, but additionally takes the commitment key ck as input.
- $\pi_{\mathsf{cp}} \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Prove}(\mathsf{pk},\mathbf{x};\mathbf{w})$: Equal to $\Pi_{\mathsf{cp}}.\mathsf{Prove}(\mathsf{pk},\{\mathbf{x},\mathsf{cm}\};\{\mathbf{u},o\},\omega)$. It is based on $\Pi.\mathsf{Prove}$, but additionally takes the commitment cm as input, and the witness includes the committed value $\mathbf{u}$ and its opening o, together with the uncommitted part ω.
- $0/1 \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Verify}(\mathsf{vk},\mathbf{x},\mathsf{cm},\pi_{\mathsf{cp}})$ Same as $\Pi.\mathsf{Verify}$, except that it additionally takes cm as input.

Commit-Carrying SNARKs (CC-SNARKs). A *Commit-Carrying SNARKs* (CC-SNARKs) [4, 14] extend the commit-and-prove approach by generating the witness commitment during proving. This helps scale to large circuits by moving expensive computations (e.g., group exponentiation) outside the circuit: the prover commits using a circuit-dependent key ck, and the resulting commitment cm is carried into the verification equation. A cc-SNARK consists of three PPT algorithms $\Pi_{\mathsf{cc}} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$.

- $(\mathsf{ck},\mathsf{pk},\mathsf{vk}) \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Setup}(1^\lambda, \mathcal{R})$: Similar to $\Pi_{\mathsf{cp}}.\mathsf{Setup}$, but additionally outputs the circuit-dependent commitment key ck together with $(\mathsf{pk}, \mathsf{vk})$.
- $(\pi_{\mathsf{cc}}, \tilde{\mathsf{cm}}) \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Prove}(\mathsf{pk},\mathbf{x};\mathbf{w},\tilde{o})$: The witness is partitioned as $\mathbf{w} := (\mathbf{u}, \omega)$ into a committed part and an uncommitted part. The prover outputs a proof π_{cc} together with a proof-dependent commitment $\tilde{\mathsf{cm}} = \mathsf{Com}.\mathsf{Commit}(\mathsf{ck},\mathbf{u};\tilde{o})$.
- $0/1 \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Verify}(\mathsf{vk},\mathbf{x},\tilde{\mathsf{cm}},\pi_{\mathsf{cc}})$: On input the verification key vk, a statement $\mathbf{x}$, a proof-dependent commitment $\tilde{\mathsf{cm}}$, and a proof π_{cc}, the verifier outputs 0(reject) or 1(accept)

3.5 Freivalds' Algorithm

Freivalds' algorithm [11, 15] is a simple yet powerful randomized method for verifying a matrix product without recomputing it. Let $\mathbf{X}, \mathbf{Y}, \mathbf{Z} \in \mathbb{F}^{n\times n}$ and consider the claim $\mathbf{X}\cdot\mathbf{Y} = \mathbf{Z}$. Instead of computing $\mathbf{XY}$ in $O(n^3)$ time, Freivalds samples a uniformly random vector $\mathbf{r} \xleftarrow{\$} \mathbb{F}^n$ and checks

$$\mathbf{X}(\mathbf{Yr}) \stackrel{?}{=} \mathbf{Zr}.$$

This verification runs in $O(n^2)$ field operations. This algorithm has soundness error: if $\mathbf{XY} = \mathbf{Z}$, it always accepts. Otherwise, when $\mathbf{XY} \neq \mathbf{Z}$, the check passes with probability at most $1/|\mathbb{F}|$, i.e.,

$$\Pr\left[\,\mathbf{X}(\mathbf{Yr}) = \mathbf{Zr} \;\middle|\; \mathbf{XY} \neq \mathbf{Z}\,\right] \le \frac{1}{|\mathbb{F}|}.$$

4 Technical Overview

The main computational bottleneck in Transformers is the quadratic (or higher) cost of the self-attention matrix multiplications. To address this, recent work [1, 7, 25, 28, 29] proposes alternatives that reduce the cost of self-attention, with several methods achieving near-linear complexity while maintaining accuracy. One important line of research is motivated by the observation that the self-attention matrix is approximately low rank. By projecting the input sequence into a lower-dimensional subspace, these methods retain the essential information while reducing the computational cost to $O(n)$.

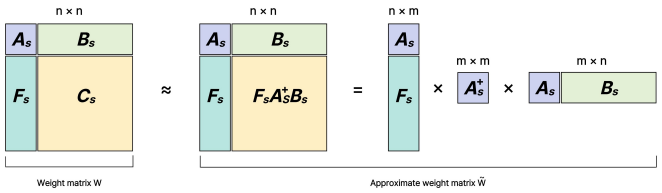
In the domain of zero-knowledge proofs, Freivalds' algorithm has been widely used to reduce the verification cost of matrix multiplication from $O(n^3)$ to $O(n^2)$. However, prior work has rarely explored how the inherent low-rank structure of self-attention can be exploited to further reduce the proof cost in zero-knowledge settings.

In this work, we propose **zkDietFormer**, which leverages the low-rank structure of self-attention to reduce proof cost to linear complexity. Our key idea is to avoid proving the full attention matrix and instead construct a zero-knowledge proof over its low-rank approximation. To address the $O(n^2)$ bottleneck and optimize proof efficiency, we examine several representative approaches:

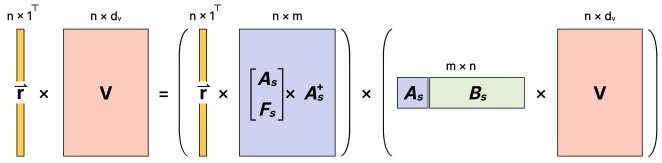
- Linformer is based on the observation that the self-attention matrix is low rank. It projects the keys and values into a lower-dimensional space using learned projection matrices, thereby achieving linear complexity.
- Nyströmformer applies the Nyström method to select representative landmark points from the full matrix and reconstruct the attention matrix from these landmarks.
- Performer uses the FAVOR+ algorithm to approximate attention via kernelization. Rather than projecting into a low-dimensional space, it decomposes attention as a product of low-dimensional matrices and rearranges computation to improve efficiency.

In our design, we adopt Nyströmformer, as it does not require introducing additional learned projection matrices into the proving

circuit, thereby simplifying the proof structure. Moreover, the theoretically grounded Nyström approximation provides numerically stable reconstruction regardless of sequence length.



(a) Nystromformer matrix approximation



(b) Our zkDiet Freivald Algorithm

Figure 1: Low rank matrix multiplication with Freivald Algorithm

As illustrated in Figure 1a, rather than directly handling the $n \times n$ self-attention matrix, we follow the Nyströmformer procedure and decompose it into a sequence of low-dimensional matrices of shapes $n \times m$, $m \times m$, and $m \times n$. By applying Freivalds' algorithm to this low-rank representation(in Figure 1b), we compress the verification workload from $O(n^2)$ to $O(n)$, effectively reducing it to vector multiplications of size $n + m^3 + nm^2 + mnd_v + m^3 + nmd_v$. This enables simultaneous reduction of both the prover's proof-generation time and the verifier's computational overhead.

To verify such low-rank matrix multiplications using Freivalds' algorithm, we adopt a Commit-and-Prove SNARK (CP-SNARK) architecture. (in Figure 2) The attention matrix is first committed outside the circuit, and the prover shows that the committed matrix is consistent with the private witness inside the circuit, while simultaneously proving that the matrix multiplication result is correct via a Freivalds algorithm with linear complexity. Instead of transmitting the full attention matrix, only the externally generated commitment is provided to the verifier, who then performs merely constant-time commitment checks. This design substantially reduces both proving and verification costs for matrix operations arising in large-scale models.

Because zero-knowledge circuits operate over finite fields, real-valued Transformer computations cannot be represented directly. We therefore quantize the computations in a manner that minimizes precision loss, leveraging the R1CS-friendly quantization technique proposed in [10].

5 zkDietFormer: Efficient Proofs of Low-Rank Matrix Multiplication via Freivalds' Algorithm

In this section, we present the core protocol of **zkDietFormer**. Our goal is to prove the correctness of Nyström -approximated low-rank self-attention while avoiding any explicit computation of the $n \times n$ attention matrix inside the circuit. Concretely, we design the protocol so that the number of constraints scales linearly with the sequence length n. To this end, we adopt a commit-and-prove SNARK architecture that pushes computationally expensive operations (e.g., large matrix multiplications and pseudoinverse computation) outside the circuit, and proves only their correctness inside the circuit using a small number of constraints.

5.1 Protocol Overview

Our protocol follows the message flow shown in Figure 3. First, the prover runs NyströmAttention on the input sequence $\mathbf{X} \in \mathbb{F}^{n \times d}$ and the model parameters $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{F}^{d \times d}$. This computation is performed outside the circuit, and thus can include heavy operations such as pseudoinverse computation. From this execution, the prover obtains the query, key, and value matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, where $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K$, and $\mathbf{V} = \mathbf{X}\mathbf{W}^V$. The prover then commits to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ and sends the resulting commitments $\widetilde{\mathrm{cm}}_Q, \widetilde{\mathrm{cm}}_K, \widetilde{\mathrm{cm}}_V$ to the verifier.

Next, the verifier samples random challenge vectors $\mathbf{r}_n \in \mathbb{F}^n$ and $\mathbf{r}_k \in \mathbb{F}^k$ for Freivalds' randomized checks, and sends them to the prover. The prover sets the public instance as

$$\mathbf{x} := (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k),$$

and generates a SNARK proof π using the proving key pk. The witness consists of a committed part

$$\mathbf{u} := (\mathbf{Q}, \mathbf{K}, \mathbf{V}),$$

and an uncommitted part

$$\omega := \left(\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V,\ \mathbf{Q}\widetilde{\mathbf{K}}^\top,\ \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top,\ \widetilde{\mathbf{Q}}\mathbf{K}^\top,\ \widetilde{A}^+\right).$$

Finally, the verifier checks the proof π against the public instance $\mathbf{x}$ and the previously received commitments $\widetilde{\mathrm{cm}}_Q, \widetilde{\mathrm{cm}}_K, \widetilde{\mathrm{cm}}_V$ using the verification key vk. The above interactive protocol can be made non-interactive via the Fiat–Shamir transformation.

5.2 Nyström -Based Low-Rank Self-Attention

Our system adopts a Nyström -based low-rank approximation for self-attention. In Nyströmformer , the query and key matrices are represented using k landmark vectors. Algorithm 1 describes the landmark selection procedure. Given $\mathbf{Q} = [\mathbf{q}_1, \ldots, \mathbf{q}_n]$ and $\mathbf{K} = [\mathbf{k}_1, \ldots, \mathbf{k}_n]$, We summarize the length-n sequence using k landmarks by partitioning it into k consecutive segments of length $\ell = n/k$ and defining each landmark as the average over the ℓ tokens in its segment. This yields $\widetilde{\mathbf{Q}} = [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_k]$ and $\widetilde{\mathbf{K}} = [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_k]$. With these landmarks, we avoid constructing the full $n \times n$ attention matrix and instead decompose the computation into three lower-dimensional matrices: $\mathbf{Q}\widetilde{\mathbf{K}}^\top \in \mathbb{F}^{n \times k}$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top \in \mathbb{F}^{k \times k}$, $\widetilde{\mathbf{Q}}\mathbf{K}^\top \in \mathbb{F}^{k \times n}$

We then apply the row-wise softmax to each of these matrices and compute

$$\tilde{F} := \mathrm{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d}}\right), \tilde{A} := \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d}}\right), \tilde{B} := \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d}}\right).$$

Here, $\tilde{A}$ is a small $k \times k$ matrix corresponding to the landmark submatrix of the original $n \times n$ attention matrix. Overall, the combination of $\tilde{F}$, $\tilde{A}$, and $\tilde{B}$ yields a Nyström approximation of the full attention map.

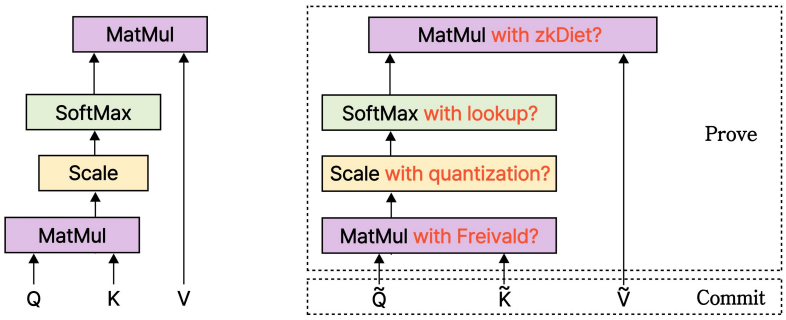


Figure 2: Overview of Self-Attention and our framework

Protocol

$\mathcal{P}(\mathsf{ck}, \mathbf{X}; \{W^Q, W^K, W^V\})$ $\qquad$ $\mathcal{V}(\mathsf{vk}, \mathbf{X})$

$\mathbf{Q}, \mathbf{K}, \mathbf{V}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \widetilde{A}^+ \leftarrow \mathsf{NyströmAttention}(\mathbf{X}, W^Q, W^K, W^V)$

$(\widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V) \leftarrow \mathsf{Commit}(\mathsf{ck}, \mathbf{Q}; o_{\tilde{Q}}),\ \mathsf{Commit}(\mathsf{ck}, \mathbf{K}; o_{\tilde{K}}),\ \mathsf{Commit}(\mathsf{ck}, \mathbf{V}; o_{\tilde{V}})$

$\widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V \longrightarrow$

$\longleftarrow \mathbf{r}_n, \mathbf{r}_k$ $\qquad$ $\mathbf{r}_n \xleftarrow{\$} \mathbb{F}^n, \quad \mathbf{r}_k \xleftarrow{\$} \mathbb{F}^k$

$\mathbf{x} := \{\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k\}$

$\mathbf{u} := \{\mathbf{Q}, \mathbf{K}, \mathbf{V}\}$

$\omega := \{W^Q, W^K, W^V, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \widetilde{A}^+\}$

$\pi \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; (\mathbf{u}, \omega))$

$\pi \longrightarrow$

$\Pi_{\mathsf{cc}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \widetilde{\mathsf{cm}}_Q, \widetilde{\mathsf{cm}}_K, \widetilde{\mathsf{cm}}_V, \pi)$

Figure 3: Our protocol

Finally, we compute an approximation to the Moore–Penrose pseudoinverse $\tilde{A}^+$ of the $k \times k$ matrix $\tilde{A}$. While one could compute $\tilde{A}^+$ via SVD, this is typically inefficient on GPU hardware. Therefore, Nyströmformer approximates $\tilde{A}^+$ via an iterative polynomial approximation to the SVD. Algorithm 2 shows this iterative procedure.

Using the resulting $\tilde{A}^+$, we form the final approximated attention matrix

$$\hat{S} := \tilde{F} \cdot \tilde{A}^+ \cdot \tilde{B},$$

and compute the attention output as

$$\mathbf{Y} := \hat{S} \cdot \mathbf{V}.$$

Algorithm 3 provides the complete Nyström -based self-attention computation used in our framework.

5.3 Efficient Verification via Freivalds' Algorithm

Our protocol uses Freivalds' algorithm as a core tool to efficiently verify that the low-rank self-attention computation is performed correctly. In particular, within the zk-SNARK circuit, we employ Freivalds' algorithm to ensure that the following key operations

Algorithm 1 Select landmarks

Input: Query $\mathbf{Q}$ and Key $\mathbf{K} \in \mathbb{F}^{n \times d}$
Output: Landmark points $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{F}^{k \times d}$

SelectLandmarks$(\mathbf{Q}, \mathbf{K})$:

1: **parse** $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n], \mathbf{K} = [\mathbf{k}_1, \mathbf{k}_2, \ldots, \mathbf{k}_n]$
2: $\ell \leftarrow n/k$
3: **for** $j \in \{1, \ldots, k\}$ **do**
4: $\quad \tilde{\mathbf{q}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{q}_i}{\ell}, \tilde{\mathbf{k}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{k}_i}{\ell}$
5: **endfor**
6: $\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1, \tilde{\mathbf{q}}_2, \ldots, \tilde{\mathbf{q}}_k]$
7: $\widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1, \tilde{\mathbf{k}}_2, \ldots, \tilde{\mathbf{k}}_k]$
8: **return** $(\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}})$

Algorithm 2 Compute Moore–Penrose inverse

Input: selected landmark matrix $\tilde{A} \in \mathbb{F}^{k\times k}$
Output: approximate pseudoinverse $\tilde{A}^{+} \in \mathbb{F}^{k\times k}$

ComputeMPInverse($\tilde{A}$):

$Z_0 \leftarrow \tilde{A}^{\top} / \left(\|\tilde{A}\|_1 \cdot \|\tilde{A}\|_\infty \right)$
Set T (e.g., $T = 6$ iterations)
for $t \in \{0, \ldots, T-1\}$ **do**
  $Z_{t+1} \leftarrow \frac{1}{4} Z_t \left(13I - \tilde{A} Z_t \left(15I - \tilde{A} Z_t \left(7I - \tilde{A} Z_t \right) \right) \right)$
endfor
$\tilde{A}^{+} \leftarrow Z_T$
return $\tilde{A}^{+}$

Algorithm 3 Nyström approximation of attention matrix

Input: Input sequence $\mathbf{X} \in \mathbb{F}^{n\times d}$,
  Model parameters $W^{Q}, W^{K}, W^{V} \in \mathbb{F}^{d\times d}$
Output: Attention matrix $\mathbf{Y} := \hat{\mathbf{S}}\mathbf{V} \in \mathbb{F}^{n\times d}$,
  Auxiliary $aux := \mathbf{Q}, \mathbf{K}, \mathbf{V}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+}$

NyströmAttention($\mathbf{X}, W^{Q}, W^{K}, W^{V}$):

$\mathbf{Q} := \mathbf{X} \cdot W^{Q}$
$\mathbf{K} := \mathbf{X} \cdot W^{K}$
$\mathbf{V} := \mathbf{X} \cdot W^{V}$
$\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \leftarrow$ SelectLandmarks($\mathbf{Q}, \mathbf{K}$)
$\tilde{F} := \sigma\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right), \tilde{A} := \sigma\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right), \tilde{B} := \sigma\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^{\top}}{\sqrt{d}} \right)$
$\tilde{A}^{+} \leftarrow$ ComputeMPInverse($\tilde{A}$)
$\hat{S} := \tilde{F} \cdot \tilde{A}^{+} \cdot \tilde{B}$
$\mathbf{Y} := \hat{S} \cdot \mathbf{V} = \tilde{F} \cdot \tilde{A}^{+} \cdot \tilde{B} \cdot \mathbf{V}$
$aux := \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+}$
return $\mathbf{Y}, aux$

can be verified with linear cost.

Query/Key/Value matrix generation. Using the verifier's random vector $\mathbf{r}_n$, we check the relation $\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q}$ by comparing $\mathbf{r}_n^{\top}\mathbf{Q}$ with $\mathbf{r}_n^{\top}(\mathbf{X}\mathbf{W}^{Q})$. We verify $\mathbf{K}$ and $\mathbf{V}$ in the same way. Rather than proving the full $n \times d$ matrix product directly, this Freivalds' check reduces verification to computing only the vector-matrix product $\mathbf{r}_n^{\top}\mathbf{X}W^{Q}$, thereby reducing the complexity to $O(n)$.

Low-rank matrix operations. We also verify low-dimensional products such as $\mathbf{Q}\widetilde{\mathbf{K}}^{\top}$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}$, and $\widetilde{\mathbf{Q}}\mathbf{K}^{\top}$ by multiplying each matrix with a random vector of matching dimension, i.e., $\mathbf{r}_n$ or $\mathbf{r}_k$. For instance, we can certify the correctness of $\mathbf{Q}\widetilde{\mathbf{K}}^{\top}$ by checking the following equality:

$$\mathbf{r}_n^{\top}(\mathbf{Q}\widetilde{\mathbf{K}}^{\top}) \stackrel{?}{=} (\mathbf{r}_n^{\top}\mathbf{Q})\,\widetilde{\mathbf{K}}^{\top}$$

Similarly, the products $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}$ and $\widetilde{\mathbf{Q}}\mathbf{K}^{\top}$ are validated with linear cost, without explicitly computing the full matrix multiplications inside the circuit.

Relation

1: $\mathcal{R}(\mathbf{x}, \mathbf{w})$:
2: **parse** $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k), \mathbf{w} = (\mathbf{u}, \omega)$
3: **parse** $\mathbf{u} = (\mathbf{Q}, \mathbf{K}, \mathbf{V})$
4: **parse** $\omega = (W^{Q}, W^{K}, W^{V}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+})$
5: **parse** $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n], \mathbf{K} = [\mathbf{k}_1, \mathbf{k}_2, \ldots, \mathbf{k}_n]$
6: $\mathbf{r}_n \cdot \mathbf{Q} = \mathbf{r}_n \cdot \mathbf{X} \cdot W^{Q}$
7: $\mathbf{r}_n \cdot \mathbf{K} = \mathbf{r}_n \cdot \mathbf{X} \cdot W^{K}$
8: $\mathbf{r}_n \cdot \mathbf{V} = \mathbf{r}_n \cdot \mathbf{X} \cdot W^{V}$
9: **for** $j \in \{1, \ldots, k\}$ **do**
10: $\tilde{\mathbf{q}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{q}_i}{\ell}, \tilde{\mathbf{k}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{k}_i}{\ell}$
11: **endfor**
12: $\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_k], \widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_k]$
13: $\mathbf{r}_n \cdot \mathbf{Q}\widetilde{\mathbf{K}}^{\top} = \mathbf{r}_n \cdot \mathbf{Q} \cdot \widetilde{\mathbf{K}}^{\top}$
14: $\mathbf{r}_k \cdot \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top} = \mathbf{r}_k \cdot \widetilde{\mathbf{Q}} \cdot \widetilde{\mathbf{K}}^{\top}$
15: $\mathbf{r}_n \cdot \widetilde{\mathbf{Q}}\mathbf{K}^{\top} = \mathbf{r}_n \cdot \widetilde{\mathbf{Q}} \cdot \mathbf{K}^{\top}$
16: $\widetilde{F} = \sigma\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right)$
17: $\widetilde{A} = \sigma\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}}{\sqrt{d}} \right)$
18: $\widetilde{B} = \sigma\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^{\top}}{\sqrt{d}} \right)$
19: $\mathbf{r}_k \cdot \widetilde{A} \cdot \widetilde{A}^{+} \cdot \widetilde{A} = \mathbf{r}_k \cdot \widetilde{A}$
20: $\mathbf{r}_k \cdot \widetilde{A}^{+} \cdot \widetilde{A} \cdot \widetilde{A}^{+} = \mathbf{r}_k \cdot \widetilde{A}^{+}$
21: $\mathbf{r}_k \cdot (\widetilde{A}\widetilde{A}^{+})^{\top} = \mathbf{r}_k \cdot \widetilde{A}\widetilde{A}^{+}$
22: $\mathbf{r}_k \cdot (\widetilde{A}^{+}\widetilde{A})^{\top} = \mathbf{r}_k \cdot \widetilde{A}^{+}\widetilde{A}$
23: $\mathbf{r}_n \cdot \mathbf{Y} = \mathbf{r}_n \cdot \widetilde{F} \cdot \widetilde{A}^{+} \cdot \widetilde{B} \cdot \mathbf{V}$

Figure 4: Relation of zkDietFormer

Moore–Penrose pseudoinverse. Computing the Moore–Penrose pseudoinverse $\tilde{A}^{+}$ inside the circuit is inefficient. Instead, we certify that $\tilde{A}^{+}$ is the pseudoinverse of $\tilde{A}$ by checking the four Moore–Penrose conditions, which correspond to lines 19–22 in Figure 4:

- $\tilde{A} \cdot \tilde{A}^{+} \cdot \tilde{A} = \tilde{A}$
- $\tilde{A}^{+} \cdot \tilde{A} \cdot \tilde{A}^{+} = \tilde{A}^{+}$
- $(\tilde{A}\tilde{A}^{+})^{\top} = \tilde{A}\tilde{A}^{+}$
- $(\tilde{A}^{+}\tilde{A})^{\top} = \tilde{A}^{+}\tilde{A}$

This allows the prover to efficiently demonstrate that $\tilde{A}^{+}$ is a valid Moore–Penrose pseudoinverse of $\tilde{A}$ without explicitly computing it in-circuit.

The relations that the prover must satisfy are formalized in Figure 4 as the definition of the circuit relation. In particular, the relation $\mathcal{R}(\mathbf{x}, \mathbf{w})$ takes the public instance $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_k)$ and the witness $\mathbf{w} = (\mathbf{Q}, \mathbf{K}, \mathbf{V}, W^{Q}, W^{K}, W^{V}, \mathbf{Q}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^{\top}, \widetilde{\mathbf{Q}}\mathbf{K}^{\top}, \widetilde{A}^{+})$, and enforces all required equalities as well as the softmax computation. All key equalities are expressed in the form of products with the random vectors, accordingly, the circuit computes only vector inner products.

6 Security

In this section, we show that zkDietFormer satisfies the completeness and security requirements. Since zkDietFormer is built upon the cc-Groth16 protocol, it inherits its guarantees of completeness, soundness, and zero-knowledge. **Completeness**. If an honest prover correctly performs the Nyströmformer-based linear attention computation and generates a proof, then the verifier always accepts it. In our system, the only potential source of completeness error arises from nonlinear operations handled via lookup tables (i.e., exponentiation operations). For matrix operations, the Freivalds' algorithm always returns true when the equation is correct. In the case of lookup tables, one may theoretically consider situations where the denominator becomes zero. However, **zkDiet** avoids this issue through quantization with zero-point adjustment, which maps all inputs into the positive domain. As a result, any scenario in which the denominator is approximated as zero is mathematically eliminated, preserving completeness.

Soundness. Even a malicious prover cannot convince the verifier of an incorrect matrix computation except with negligible probability. Our system satisfies two essential security properties.

First, the integrity of the matrix multiplication $AB = C$ is guaranteed by Freivalds' algorithm. When $AB \neq C$, the probability that a randomly chosen vector r satisfies $ABx = C$ is extremely small, bounded by $O(1/|\mathbb{F}|)$, which is inversely proportional to the size of the finite field $\mathbb{F}$. To simplify the analysis, we assume in the following proof that the random vector r used in Freivalds' algorithm is sampled from the binary domain $\{0,1\}^n$.

Proof. Since $\hat{D}^{-1}$ is a diagonal matrix, the overall complexity $O(nkd)$ is determined by the $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V})$ computation, excluding the cost of forming $\hat{D}^{-1}$. In the following section, we demonstrate that the Freivalds' algorithm correctly verifies the computation of $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V})$. Suppose the prover outputs a matrix y' that has a different **structure or value** from the correct output $y = \tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V})$. Let

$$s_1' = \vec{r}\, y', \qquad s_2 = (\vec{r}\, \tilde{\mathbf{F}}\mathbf{A}'^{+})((\mathbf{B}')^{\top}\mathbf{V}).$$

where $\tilde{\mathbf{F}}\mathbf{A}'^{+}, \mathbf{B}' \in \mathbb{R}^{n\times k}, \mathbf{V} \in \mathbb{R}^{n\times d}, \vec{r} \in \{0,1\}^n$. Let ϵ denote the error probability that the Freivalds verification outputs TRUE even when it is incorrect. We claim that if $\tilde{\mathbf{F}}\mathbf{A}'^{+} \times ((\mathbf{B}')^{\top})\mathbf{V} = y'$, then $\epsilon = 0$, and if $\tilde{\mathbf{F}}\mathbf{A}'^{+} \times ((\mathbf{B}')^{\top})\mathbf{V} \neq y'$, then $\epsilon \leq \frac{1}{2}$.

- **Case** $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) = y'$ **:** The error matrix $D = \tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) - y'$ is $0_{n\times d}$. Regardless of the choice of the vector $\vec{r}$, the equality is always satisfied because $\vec{r}D = \vec{0}$. Hence,

$$\epsilon = \Pr[\vec{r}D \neq 0] = \Pr[\vec{r}0 \neq \vec{0}] = 0$$

  Therefore, the probability of error is $\epsilon = 0$.

- **Case** $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) \neq y'$ **:** Let $\vec{P} = \vec{r}D = (p_1, p_2, ..., p_d)$, where $D = \tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) - y'$. Since $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) \neq y'$, there exists at least one nonzero element in D. Let $d_{i^*,j}$ be the first nonzero entry in column j. For the value $\Delta = \sum_{k\neq i^*} r_k d_{kj}$. Using Bayes' Theorem, we can partition over Δ.

$$p_j = \sum_{k=1}^{n} r_k d_{kj} = r_{i^*} d_{i^*j} + \sum_{k\neq i^*} r_k d_{kj} = r_{i^*} d_{i^*j} + \Delta. \tag{1}$$

$$\begin{aligned}\Pr[p_j = 0] = \Pr[p_j = 0|\Delta = 0] \cdot \Pr[\Delta = 0] \\ + \Pr[p_j = 0|\Delta \neq 0] \cdot \Pr[\Delta \neq 0].\end{aligned} \tag{2}$$

$$\Pr[p_j = 0|\Delta = 0] = \Pr[r_{i^*} = 0] = \frac{1}{2}. \tag{3}$$

For $\Pr[p_j = 0|\Delta \neq 0]$, if $r_{i^*} d_{i^*j} + \Delta = 0$, it holds as long as $r_{i^*} = 1$ (among the possible values of $r_{i^*} \in \{0,1\}$)so

$$\Pr[p_j = 0|\Delta \neq 0] = \Pr[r_{i^*} = 1] = \frac{1}{2}$$

if $d_{i^*j} + \Delta \neq 0$, then r_{i^*} cannot take any value to satisfy the condition, so $\Pr[p_j = 0|\Delta \neq 0] = 0$. Therefore, $\Pr[p_j = 0|\Delta \neq 0] \leq \frac{1}{2}$.
After taking this case into account, the probability can be rewritten as follows:

$$\begin{aligned}\Pr[p_j = 0] &\leq \frac{1}{2} \cdot \Pr[\Delta = 0] + \frac{1}{2} \cdot \Pr[\Delta \neq 0] \\ &= \frac{1}{2} \cdot \Pr[\Delta = 0] + \frac{1}{2} \cdot (1 - \Pr[\Delta = 0]) \\ &= \frac{1}{2}.\end{aligned} \tag{4}$$

$$\epsilon = \Pr[\vec{P} = 0 \mid y' \neq y]. \tag{5}$$

$$\begin{aligned}\epsilon &= \Pr[\vec{P} = 0] \\ &= \Pr[p_1 = 0 \wedge \cdots \wedge p_j = 0 \wedge \cdots \wedge p_d = 0] \\ &\leq \Pr[p_j = 0] \leq \frac{1}{2}.\end{aligned} \tag{6}$$

$$\epsilon \leq \frac{1}{2}. \tag{7}$$

After t independent repetitions, the ϵ becomes $(\frac{1}{2})^t$, which is negligible in t. Therefore, the verification protocol is sound: the probability that it outputs TRUE when $\tilde{\mathbf{F}}\mathbf{A}'^{+}((\mathbf{B}')^{\top}\mathbf{V}) \neq y'$ is $\epsilon \leq (\frac{1}{2})^t$. □

Second, the strong binding property of the KZG polynomial commitment ensures that the prover cannot claim values different from those already committed. Using the commit-and-prove capability of CP-SNARK, we prove (1) correctness of matrix multiplication and (2) consistency of the Softmax outputs, while cp-link additionally guarantees (3) input consistency across these two computations. Consequently, any attempt to replace a committed value with another one cannot pass verification.

Zero-Knowledge. **zkDiet** reveals no secret information during inference, including model weights, inputs, or intermediate activations. All values are handled in committed form. The verifier learns only that the linear transformer computation was carried out correctly, while gaining no additional information about the committed values.

7 Experiments

In this section, we evaluate the concrete proof generation costs of **zkDietFormer** and compare them against a standard self-attention baseline. Because our contribution targets the *architectural* cost of proving attention—i.e., the circuit structure rather than the learned weights—the evaluation focuses on the cost of the ZK circuit itself. We use modified HuggingFace BERT and Nystromformer configurations (modified hidden dimensions and layer counts) to enable proving across multiple model sizes; consequently, model inference accuracy is not a focus of this evaluation.

7.1 Implementation

zkDietFormer is implemented as a hybrid system consisting of a Python-based witness generation pipeline and a Rust-based zero-knowledge proving circuit. Witness data (model weights and intermediate activations) are generated using PyTorch 3.11.15 and HuggingFace Transformers 5.8.0, which instrument the standard `BertForMaskedLM` and `NystromformerForMaskedLM` model classes with forward hooks to capture per-layer activations. The Rust circuit is built on the arkworks library over the BLS12-381 elliptic curve and uses the BLAKE2s hash function for cryptographic operations. The implementation is compiled with rustc 1.97.0-nightly.

The proving system used the LegoGroth16 [4] as a commit-and-prove SNARK (CP-SNARK) for linking committed values across sub-circuits. Non-arithmetic operations—softmax exponentials, SigmoidGELU activation, and LayerNorm inverse square root—are evaluated using Lasso [21] lookup arguments with precomputed lookup tables indexed by quantized inputs.

Activation Function Replacement. Both BERT and Nyströmformer use GELU [13] as their default activation function, typically implemented via the approximation $\mathrm{GELU}(x) \approx 0.5\,x\big(1+\tanh\big[\sqrt{2/\pi}\,(x+0.044715\,x^3)\big]\big)$. We replace GELU in both models with *SigmoidGELU*, defined as

$$\mathrm{SigmoidGELU}(x) = x \cdot \sigma(1.702\,x),$$

where $\sigma(\cdot)$ denotes the logistic sigmoid. This alternative, originally noted by Hendrycks and Gimpel [13] as a close approximation to GELU, is substantially more circuit-friendly because the sigmoid function can be evaluated via a single lookup table, whereas the tanh-based form requires a more involved computational procedure. The replacement is applied to all layers in both models.

Pseudoinverse Verification. The Nyströmformer attention mechanism requires computing the Moore–Penrose pseudoinverse $K^\dagger$ of a small $m \times m$ matrix K derived from the softmax of landmark interactions, where m is the number of landmarks. The reference implementation [29] computes this pseudoinverse via the iterative Newton–Schulz method: starting from an initial estimate $V_0 = \|K\|_1^{-1} K^\top$, the iteration

$$V_{i+1} = \tfrac{1}{4} V_i \big(13\,I - KV_i\big(15\,I - KV_i\big(7\,I - KV_i\big)\big)\big)$$

is applied for, for example, $n_{\text{iter}} = 6$ rounds. Directly arithmetizing this cubic Newton iteration is highly expensive: each round requires three sequential matrix multiplications, yielding 18 matrix multiplications in total per layer, all of which would need to be encoded as R1CS constraints.

Instead of proving the entire iterative computation in-circuit, we adopt a witness-supplied verification approach: the prover computes $K^\dagger$ outside the circuit using the Newton–Schulz iteration as in the reference implementation, then commits the result V as a witness. The circuit then verifies that V is indeed the pseudoinverse of K by checking the four defining Moore–Penrose conditions, using the Freivalds-based verification introduced in Section 5.3. Since $m = 32$ is small and independent of the sequence length n, this verification adds only a constant overhead regardless of input length.

Quantization. Zero-knowledge circuits operate over finite fields and cannot represent floating-point arithmetic directly. We therefore quantize all model computations following the R1CS-friendly quantization scheme of ZEN [10]. All weights and activations are converted to 8-bit asymmetric fixed-point representations, and the circuit precision parameter is set to $k_{\text{bits}} = 22$, providing sufficient headroom for accumulated multiplication errors across a full layer. Different operators use different fractional precisions to maintain numerical fidelity: LayerNorm uses 8-bit fractional precision (`FRAC_BITS` = 8); Softmax uses 16-bit fractional precision (`FRAC_BITS` = 16), reflecting the higher dynamic range required by the exponential function; and GELU/Sigmoid are evaluated via lookup tables of size $2^{16} = 65{,}536$ entries, embedded into the circuit using lookup arguments [21].

7.2 Experimental Configuration

Hardware. All experiments are conducted on a Mac Studio equipped with an Apple M1 Ultra chip (20 cores: 16 performance + 4 efficiency) and 128 GB of unified memory, running macOS 14.6.1. No GPU acceleration is used, as the underlying ZK library operates exclusively on CPU. All reported timing measurements are CPU wall-clock time using Rayon's default thread pool, which utilizes all available hardware threads (20 cores on our platform). Peak memory is measured as the maximum resident set size (RSS) via the system `time` utility.

Baselines. We adopt two publicly available Transformer architectures from the HuggingFace Transformers library [27]:

- **BERT** (`bert-base-uncased`, Devlin et al. [9]): employs standard scaled dot-product self-attention with $O(n^2 d)$ complexity.
- **Nyströmformer** (`uw-madison/nystromformer-512`, Xiong et al. [29]): replaces the full attention matrix with a Nyström low-rank approximation, reducing attention complexity to $O(n)$. The pretrained checkpoint uses a maximum position embedding length of 512 (effective length 510).

Both models are instantiated using the standard HuggingFace classes (`BertForMaskedLM` and `NystromformerForMaskedLM`) without any custom architectural modifications beyond the activation replacement and quantization.

Random Weight Initialization. Because circuit complexity is entirely determined by the model *architecture*—the computation graph and tensor dimensions—and is independent of the actual weight values, we do not use pretrained weights. Instead, we randomly initialize both `BertForMaskedLM` and `NystromformerForMaskedLM` from their respective HuggingFace configurations. This also allows us to override the `max_position_embeddings` parameter to match each target sequence length n, enabling measurements beyond the

original 512-token pretraining limit. We emphasize that this choice does not affect the validity of our results: the number of R1CS constraints is a function of n, d, and the attention mechanism, not of the weight matrices $\mathbf{W}$.

Model Configurations. We evaluate five configurations that span the standard BERT model family, from the smallest (Tiny) to full-scale (Base):

Model	L	d	A
Tiny	2	128	2
Mini	4	256	4
Small	4	512	8
Medium	8	512	8
Base	12	768	12

Table 1: Model configurations used in experiments. L is the number of Transformer layers, d is the hidden dimension, and A is the number of attention heads.

For each model scale, the BERT baseline and Nyströmformer variant share identical structural parameters (L, d, A), differing only in the attention kernel.

7.3 Results

All reported metrics are *per-layer*: each table entry corresponds to a single attention or MLP layer. The full-model proving cost is obtained by multiplying per-layer cost by L. We measure six sequence lengths $n \in \{128, 256, 512, 1024, 2048, 4096\}$ with a fixed landmark count m=32 for Nyström attention.

For each circuit instance, we report constraint count, setup time, prove time, verify time, and peak memory. Runs that exceed available memory are reported as out-of-memory (OOM); these patterns are themselves informative results, as they indicate the practical scalability boundary of each circuit variant.

Table 2 compares the per-layer attention costs between Nyström + Freivalds and BERT + Freivalds for the Tiny configuration (L=2, d=128, A=2). Each column reports aggregated metrics: setup (Groth16 + CP-link setup), prove (Groth16 prove + Lasso lookups + CP-link prove), verify (Groth16 verify + CP-link verify), and peak memory. Entries marked "—" denote runs that exceeded the 128 GB memory limit (OOM).

Tables 2 and 3 present the same data in two layouts for comparison. Table 2 reveals two key findings. First, the quadratic scaling of BERT attention in ZK circuits is clearly visible: at n=1024, BERT + Freivalds requires 1,452.9 s setup and 299.6 s proving, compared to 108.1 s and 19.8 s for Nyström —13.4× and 15.1× ratios, respectively. Second, memory consumption diverges sharply: BERT reaches 117.6 GB at n=1024 (near the 128 GB limit), while Nyström uses only 69.1 GB even at n=4096.

MLP costs. The MLP + LayerNorm sublayer is shared between both architectures and produces identical circuits. Table 4 reports per-layer MLP costs for the Tiny configuration. MLP dominates the per-layer budget: at n=1024, MLP proving takes 450.3 s versus 19.8 s for Nyström attention—22.7× more. The MLP block is also the memory bottleneck, consuming 118.4 GB at n=1024 and hitting OOM at $n \geq 2048$, whereas Nyström + Freivalds attention remains feasible up to n=4096 at 69.1 GB. In practice, the MLP—not the attention mechanism—determines the maximum provable sequence length. A complete Tiny layer at n=1024 costs approximately 906 s setup and 470 s proving; the full two-layer model requires ~2,752 s (~46 min).

Table 2: Per-layer attention costs for the Tiny configuration (L=2, d=128, A=2). Nyström + Freivalds completes all sequence lengths up to n=4096, while BERT + Freivalds hits OOM at $n \geq 2048$.

n	Method	Setup (s)	Prove (s)	Verify (ms)	Mem (GB)
128	Nyström + Freiv.	17.1	6.9	216	3.7
	BERT + Freiv.	25.0	4.5	146	4.9
256	Nyström + Freiv.	33.3	9.3	349	7.2
	BERT + Freiv.	91.6	13.4	274	17.2
512	Nyström + Freiv.	54.0	12.4	372	11.1
	BERT + Freiv.	380.5	50.0	548	58.4
1024	Nyström + Freiv.	108.1	19.8	643	21.0
	BERT + Freiv.	1,452.9	299.6	1,098	117.6
2048	Nyström + Freiv.	207.5	35.1	700	35.5
	BERT + Freiv.		— (OOM)		
4096	Nyström + Freiv.	446.6	68.9	1,244	69.1
	BERT + Freiv.		— (OOM)		

Table 3: Same data as Table 2 (alternative layout): Nyström + Freivalds results listed first for all n, followed by BERT + Freivalds.

Method	n	Setup (s)	Prove (s)	Verify (ms)	Mem (GB)
Nyström + Freiv.	128	17.1	6.9	216	3.7
	256	33.3	9.3	349	7.2
	512	54.0	12.4	372	11.1
	1024	108.1	19.8	643	21.0
	2048	207.5	35.1	700	35.5
	4096	446.6	68.9	1,244	69.1
BERT + Freiv.	128	25.0	4.5	146	4.9
	256	91.6	13.4	274	17.2
	512	380.5	50.0	548	58.4
	1024	1,452.9	299.6	1,098	117.6
	2048		— (OOM)		
	4096		— (OOM)		

Table 4: Per-layer MLP + LayerNorm costs for the Tiny configuration (L=2, d=128). Costs are identical for Nyström and BERT architectures. OOM at $n \geq 2048$.

n	Constraints	Setup (s)	Prove (s)	Verify (ms)	Mem (GB)
128	1.0M	92.6	28.8	183	28.3
256	2.0M	190.4	58.9	187	56.5
512	3.9M	383.7	131.1	368	109.8
1024	7.8M	797.9	450.3	386	118.4
2048			— (OOM)		

7.4 Freivalds Verification Benefit

Tables 5 and 6 compare naive matrix multiplication against Freivalds randomized verification for Nyström and BERT attention circuits, respectively. In both tables, the Freivalds column consistently achieves lower constraint counts, faster setup, and faster proving, while naive circuits hit OOM at shorter sequences.

Table 5: Nyström attention: naive vs. Freivalds (Tiny, per-layer). OOM entries (—) indicate memory exhaustion at 128 GB.

n	Method	Constraints	Setup (s)	Prove (s)
128	Naive	3.1M	57.3	15.1
	Freivalds	0.5M	17.1	6.9
256	Naive	5.8M	108.8	23.9
	Freivalds	1.0M	33.3	9.3
512	Naive	11.1M	209.5	44.1
	Freivalds	1.8M	54.0	12.4
1024	Naive	21.8M	414.0	90.0
	Freivalds	3.5M	108.1	19.8
2048	Naive	43.1M	843.9	275.9
	Freivalds	6.8M	207.5	35.1
4096	Naive	— (OOM)		
	Freivalds	13.5M	446.6	68.9

Table 6: BERT attention: naive vs. Freivalds (Tiny, per-layer). Standard $O(n^2)$ attention causes quadratic constraint growth; naive verification exacerbates the effect.

n	Method	Constraints	Setup (s)	Prove (s)
128	Naive	4.8M	96.5	17.7
	Freivalds	0.7M	25.0	4.5
256	Naive	19.1M	400.4	76.5
	Freivalds	2.7M	91.6	13.4
512	Naive	— (OOM)		
	Freivalds	10.4M	380.5	50.0
1024	Naive	— (OOM)		
	Freivalds	40.6M	1,452.9	299.6
2048	Naive	— (OOM)		
	Freivalds	— (OOM)		
4096	Naive	— (OOM)		
	Freivalds	— (OOM)		

Across all measured sequence lengths, Freivalds verification reduces the constraint count by a consistent factor of ~5.8–6.3× for Nyström and ~6.6–7.1× for BERT circuits. This reduction is multiplicative on top of the asymptotic improvement from Nyström attention. Comparing the two extremes at n=256: BERT + Naive requires 19.1M constraints while Nyström + Freivalds needs only 1.0M—a combined ~20× reduction that translates to 400.4 s vs. 33.3 s in setup time.

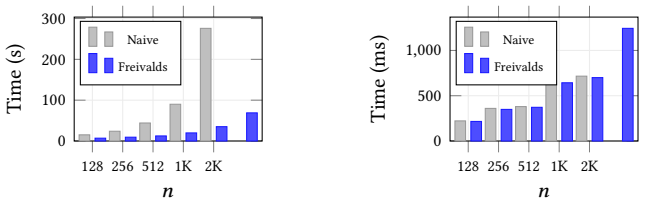
The OOM failure pattern provides direct scalability evidence. On our 128 GB machine, for the Nyström + Freivalds circuit (maximum feasible n):

- Tiny (d=128): n=4096 | Mini (d=256): n=4096
- Small (d=512): n=4096 | Medium (d=512): n=4096
- Base (d=768): n=2048

In contrast, BERT + Naive reaches only n=256 at Tiny and n=128 at Base. Within the same memory budget, Nyström + Freivalds supports sequences 16× longer than BERT + Naive.

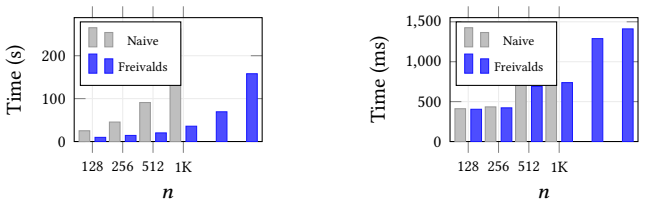
7.5 Multi-Spec Scaling

Figures 5–9 visualize the Freivalds optimization benefit for Nyström attention via per-sequence-length bar charts comparing naive and Freivalds prove time and verify time across all five model configurations.



(a) Prove time (s) **(b) Verify time (ms)**

Figure 5: Tiny Nyström attention (L=2, d=128): naive vs. Freivalds per-layer prove and verify time. Naive bars are absent at n=4096 due to OOM. The prove-time gap widens with n: at n=2048, naive (275.9 s) is 7.9× that of Freivalds (35.1 s).

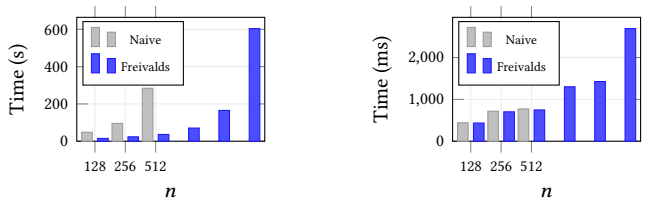


(a) Prove time (s) **(b) Verify time (ms)**

Figure 6: Mini Nyström attention (L=4, d=256): naive vs. Freivalds per-layer prove and verify time. Naive OOMs at n≥2048. At n=1024, naive (262.5 s) is 7.4× that of Freivalds (35.7 s).

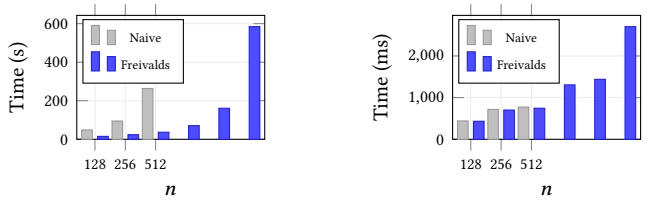
The bar charts across all five configurations (Figures 5–9) consistently demonstrate the Freivalds optimization benefit for Nyström attention. The prove-time reduction ratio grows with sequence length, reaching 5.7–7.9× at the longest feasible n for each configuration. As model dimensions increase, the naive circuit OOMs at progressively shorter sequences—naive reaches n=2048 at Tiny (d=128) but only n=256 at Base (d=768)—while Freivalds extends the feasible range to n=4096 for d≤512 and n=2048 for d=768.

An important observation from the Small and Medium figures is that per-layer costs are nearly identical when d is the same (d=512),



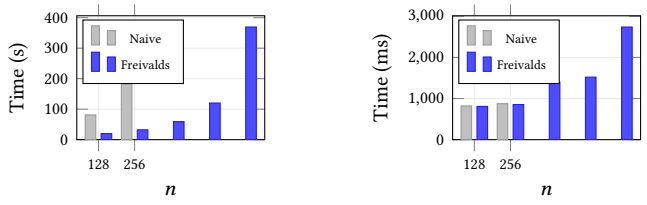
(a) Prove time (s) (b) Verify time (ms)

Figure 7: Small Nyström attention (L=4, d=512): naive vs. Freivalds per-layer prove and verify time. Naive OOMs at n≥1024. At n=512, naive prove (284.1 s) is 7.8× that of Freivalds (36.4 s).



(a) Prove time (s) (b) Verify time (ms)

Figure 8: Medium Nyström attention (L=8, d=512): naive vs. Freivalds per-layer prove and verify time. Per-layer costs are nearly identical to Small (d=512), confirming that per-layer cost depends on d, not L.



(a) Prove time (s) (b) Verify time (ms)

Figure 9: Base Nyström attention (L=12, d=768): naive vs. Freivalds per-layer prove and verify time. Naive OOMs at n≥512; Freivalds completes up to n=2048. At n=256, naive (183.1 s) is 5.7× that of Freivalds (32.3 s).

regardless of the number of layers L. This confirms that per-layer circuit complexity depends on hidden dimension and sequence length, not model depth. The total model proving cost thus scales linearly with L.

Verify time remains below 3 s across all configurations, confirming that Groth16 verification cost is negligible relative to proving regardless of model scale.

7.6 Discussion

Practical impact. Our measurements across all five configurations demonstrate that the combination of Nyström low-rank attention and Freivalds randomized matrix-multiplication verification yields multiplicative constraint savings that translate directly into longer provable sequences. At Base scale (d=768), Nyström + Freivalds proves attention up to n=2048 (40.9M constraints, 1,429 s setup), while BERT + Naive is limited to n=128 (28.7M constraints)—a 16× gap in sequence length. Even at BERT-base dimensions, a single attention layer proves in under 370 s with Nyström + Freivalds.

Memory as the practical bottleneck. All failed runs terminated due to memory exhaustion during the Groth16 trusted setup phase, which materializes the full constraint system in RAM. The peak RSS for every failed run reached ~118–122 GB (the effective usable limit on our 128 GB machine). The primary scaling bottleneck is therefore the memory footprint of the trusted setup, not prover computation time. The MLP circuit is especially memory-intensive: at d=512 and above, MLP is OOM at all sequence lengths, even n=128. At d=256 (Mini), MLP fits only at n≤256, consuming 98–122 GB and requiring 117–397 s proving per layer. Streaming setup, distributed proving, or GPU-accelerated SNARK libraries could alleviate this limitation.

Scaling to larger models. All five configurations exhibit consistent scaling behavior: the Freivalds reduction ratio (~5.7–7.9× in prove time at the longest feasible n) holds across d=128 to d=768. An important finding is that per-layer costs depend on hidden dimension d and sequence length n, not on model depth L: the Small (L=4, d=512) and Medium (L=8, d=512) configurations produce nearly identical per-layer costs. At Base scale (L=12, d=768) with Nyström + Freivalds, a single attention layer at n=2048 completes with 41.0M constraints, 1,429 s setup, and 121 GB peak memory. Proving all 12 layers of a full BERT-base model at n=2048 would require $\sim 12 \times 370 \approx 4{,}432$ s (~74 min) for attention alone, making it feasible on a single machine if layers are proved sequentially. However, MLP circuits at this scale remain out of reach without distributed proving infrastructure.

Limitations. First, all measurements use a single-machine CPU-only setup; GPU-accelerated or distributed proving may shift relative costs. Second, we do not evaluate the accuracy impact of the Nyström approximation or SigmoidGELU substitution, as these are orthogonal to circuit cost. Third, the Groth16-based system requires a trusted setup whose cost grows with circuit size; transparent SNARKs (e.g., FRI or IPA) would eliminate this at the expense of larger proofs. Finally, Groth16 verification time remains below 3 s even at Base scale with n=2048, confirming that verification cost is not a differentiating factor across model scales.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv:2004.05150 [cs.CL] https://arxiv.org/abs/2004.05150

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[3] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. 2020. Lunar: a Toolbox for More Efficient Universal and Updatable zkSNARKs and Commit-and-Prove Extensions. Cryptology ePrint Archive, Paper 2020/1069. https://eprint.iacr.org/2020/1069

[4] Matteo Campanelli, Dario Fiore, and Anaïs Querol. 2019. LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs. Cryptology ePrint

Archive, Paper 2019/142. https://doi.org/10.1145/3319535.3339820

[5] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 560–574. https://doi.org/10.1145/3627703.3650088

[6] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. arXiv:1904.10509 [cs.LG] https://arxiv.org/abs/1904.10509

[7] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. 2022. Rethinking Attention with Performers. arXiv:2009.14794 [cs.LG] https://arxiv.org/abs/2009.14794

[8] Mingshu Cong, Tsz Hon Yuen, and Siu-Ming Yiu. 2024. zkMatrix: Batched Short Proof for Committed Matrix Multiplication. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) *(ASIA CCS '24)*. Association for Computing Machinery, New York, NY, USA, 289–305. https://doi.org/10.1145/3634737.3645003

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] https://arxiv.org/abs/1810.04805

[10] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences. Cryptology ePrint Archive, Paper 2021/087. https://eprint.iacr.org/2021/087

[11] Rusins Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *IFIP Congress*. https://api.semanticscholar.org/CorpusID:45405368

[12] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/6048ff4e8cb07aa60b6777b6f7384d52-Paper.pdf

[13] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear Units (GELUs). arXiv:1606.08415 [cs.LG] https://arxiv.org/abs/1606.08415

[14] Byeongjun Jang, Gweonho Jeong, Hyuktae Kwon, Hyunok Oh, and Jihye Kim. 2024. VECTIS: Efficient Batching Framework for Group-based CP-SNARKs. Cryptology ePrint Archive, Paper 2024/1405. https://eprint.iacr.org/2024/1405

[15] Ivan Korec and Jiří Wiedermann. 2014. Deterministic Verification of Integer Matrix Multiplication in Quadratic Time. In *SOFSEM 2014: Theory and Practice of Computer Science*, Viliam Geffert, Bart Preneel, Branislav Rovan, Július Štuller, and A. Min Tjoa (Eds.). Springer International Publishing, Cham, 375–382.

[16] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. https://doi.org/10.1109/TDSC.2023.3348760

[17] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21)*. Association for Computing Machinery, New York, NY, USA, 2968–2985. https://doi.org/10.1145/3460120.3485379

[18] Meta. 2024. Llama 4 Model Card. GitHub repository documentation. https://github.com/meta-llama/llama-models/blob/main/models/llama4/MODEL_CARD.md Accessed: 2025-12-31.

[19] OpenAI. 2025. GPT-5 Model. OpenAI API Documentation. https://platform.openai.com/docs/models/gpt-5 Accessed: 2025-12-31.

[20] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. 2019. Compressive Transformers for Long-Range Sequence Modelling. arXiv:1911.05507 [cs.LG] https://arxiv.org/abs/1911.05507

[21] Srinath Setty, Justin Thaler, and Riad Wahby. 2023. Unlocking the lookup singularity with Lasso. Cryptology ePrint Archive, Paper 2023/1216. https://eprint.iacr.org/2023/1216

[22] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. arXiv:2404.16109 [cs.LG] https://arxiv.org/abs/2404.16109

[23] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *Advances in Cryptology – CRYPTO 2013*, Ran Canetti and Juan A. Garay (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 71–89.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] https://arxiv.org/abs/1706.03762

[25] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-Attention with Linear Complexity. arXiv:2006.04768 [cs.LG] https://arxiv.org/abs/2006.04768

[26] Jiasi Weng, Jian Weng, Gui Tang, Anjia Yang, Ming Li, and Jia-Nan Liu. 2023. pvCNN: Privacy-Preserving and Verifiable Convolutional Neural Network Testing. arXiv:2201.09186 [cs.CR] https://arxiv.org/abs/2201.09186

[27] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 38–45.

[28] Jeffrey T. H. Wong, Cheng Zhang, Xinye Cao, Pedro Gimenes, George A. Constantinides, Wayne Luk, and Yiren Zhao. 2025. A3 : an Analytical Low-Rank Approximation Framework for Attention. arXiv:2505.12942 [cs.CL] https://arxiv.org/abs/2505.12942

[29] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. 2021. Nyströmformer: A Nyström-Based Algorithm for Approximating Self-Attention. arXiv:2102.03902 [cs.CL] https://arxiv.org/abs/2102.03902

[30] Yancheng Zhang, Mengxin Zheng, Xun Chen, Jingtong Hu, Weidong Shi, Lei Ju, Yan Solihin, and Qian Lou. 2025. zkVC: Fast Zero-Knowledge Proof for Private and Verifiable Computing. arXiv:2504.12217 [cs.CR] https://arxiv.org/abs/2504.12217